

Software Project Management Plan

WORD LADDER

Change only one letter at a time to make a new word for each step.

Fail

Bail

Ball

Bald

Bold

Cold

Jumble Word Puzzle

Unscramble these Jumbles, one letter to each square, to form ordinary words.

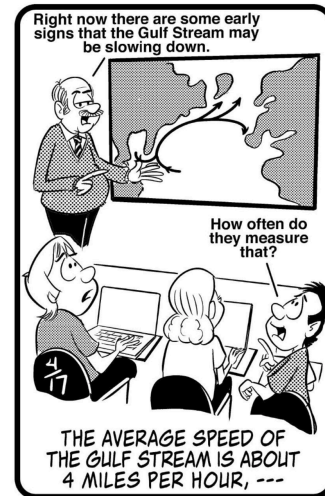
NCUDE

NRILE

VYRSUE

TRWMHA

Arrange the circled letters to form the bonus answer, as suggested by the caption in the cartoon



Word Games

FLEX

Project Manager: Daylin Jackson

Team: Luke Hayden

Zamiyah Hill

Dillon McCants

1 Overview	2
1.1 Project Summary	2
1.2 Hard Constraints	3
2 Definitions	3
3 Project Organization	3
3.1 External Interfaces	3
3.2 Internal Structure	3
3.3 Roles and Responsibilities	4
4 Managerial Process Plans	6
4.1 Start-up Plan	6
4.2 Work Plan	6

4.3 Control Plan	6
4.4 Risk Management Plan	7
4.5 Closeout Plan	8
5 Technical Process Plans	8
5.1 Process Model	8
5.2 Methods, Tools, and Techniques	8
5.3 Infrastructure Plan	9
5.4 Product Acceptance Plan	9
6 Supporting Process Plans	9
6.1 Configuration Management Plan	9
6.2 Documentation Plan	9
6.3 Quality Assurance Plan	10
6.4 Reviews and Audits	10
7 Additional Plans	10

1 Overview

1.1 Project Summary

Our project involves creating 2 word games. The first of which is called Word Ladders. And the other is called Jumble. Each game will give the user a randomly generated design with nice music playing in the background. The user will be able to choose the difficulty of either 2 of the games. Word Ladder will have a starting word at the top and an ending word at the bottom. The user will have to figure out the many words using all the letters to their disposal. However, they will have lives so that the game is challenging. The user may be given a hint off to the side so that they are not blindly putting random letters. Word Jumble will have the user unjumbling an assortment of different digit letters. Each assortment must be unjumbled into a word to win the game.

Client Deliverables

- The client wants different themes.
- Themes that can be changed by the user.
- And the themes are relative to the words. For example, the theme is fall and the word is pumpkin.
- Then the client also wants difficulty levels: easy, medium, hard.
- A limit to the amount of words there are in the game.

Project Milestone

- Completed our MOU
- Finished first biweekly report

- We are going to begin the planning phase first, we aim to get the first prototype, 1st done by after the midterm, once that's finished we should be able to get through with the 2nd sprint fairly quickly unless we decide to spend the extra time with better designs. In between those are gonna be our obvious planning phases and designing periods.

1.2 Hard Constraints

In this class we are limited to only using the programming language Java. Some of the group have never used Java to code so it will be a learning experience. We are required to use the Agile lifecycle model which loops between iterations of our project. The team must implement the use of Object Oriented Design instead of Structural Analysis and Design Technique. Objected Oriented Design(OOA/D) relies on different diagrams. OOA/D is built to show the relationship between classes. We will apply the Model-View-Controller(MVC) which divides applications into data, user interface, and input. This is helpful because it is easier to add new features and allows each component to be tested in isolation to the other along with other features. JavaDocs will help add structured comments so that not only the team understands, but the client as well.

2 Definitions

Objected Oriented Design and Analysis (OOA/D)
UI (User Interface)

3 Project Organization

3.1 External Interfaces

- The client (Mr. Santa Cruz)
- Classmates
- Stakeholders
- Users
- South Alabama
- Testers

3.2 Internal Structure

- Project Manager
- Development Lead
- Team Scribe

- Requirements Analyst
- Testers
- Development Team

Overall, we will work together as a single development team led by Daylin Jackson. We will communicate using GroupMe and our allotted time during labs. Our development activities will be done through a repository we have shared on GitHub, following an iterative development approach through the Agile lifecycle model.

3.3 Roles and Responsibilities

- Project Manager - Daylin Jackson, overseeing the entire project, mainly in charge of tracking our groups progress and coordinating with our client.
- Development Lead - Dillon McCants, overseeing development and is the primary programmer for the project.
- Team Scribe - Zamiyah Hill, she is maintaining the documentation for the project and records the notes we take during meetings. She will also ensure that all our documentation is accurate and complete.
- Requirements Analyst - Luke Hayden, focused on analyzing the requirements given from the client during meetings. Will ensure that the application has all the necessary requirements that both the client and users want.
- Testers - Entire group, testing the project and code as we work through to ensure it has no bugs and works as intended.
- Development Team - Entire group, we are all working together on the project, each doing whatever we can to aid our primary programmer, Dillon McCants.

● = Primary Skill ○ = Secondary Skill △ = Interest

Task / Skill	Daylin	Dillon	Zamiyah	Luke
Project Management	●		○	
Java Programming	△	●		○
UI Design	△	○		
Requirements Analysis	○			●
Documentation			●	△
Testing	△	△		●
GitHub Control		○		○

Client Communication	•			○
-------------------------	---	--	--	---

4 Managerial Process Plans

4.1 Start-up Plan

Everyone in the group agreed upon using Github as the repository. Dillon made the GitHub and invited everyone in the group to join. Afterwards, he sent the repository to the client so that he could look at the work.

4.2 Work Plan

Tasks to be performed are making sure to interact with the client and realize our resources so that we can make time to finish iterations of the project before the deadline. We first started with the Requirement Elicitation, which is already finished. We are now in our Analysis phase, we're currently in the midst of forming what the client wants into organized diagrams and charts. Then we will move on to System Design, designing the structure of the project. Afterwards, we'll move on to the Object Design in which we form how the classes will work in the project. Then we'll implement everything with coding iterations. If we have to go back and make tweaks to the designs, then we will do so. And finally we'll run multiple test iterations where our Implementation and Testing will run hand in hand as we form the most ideal system for our client.

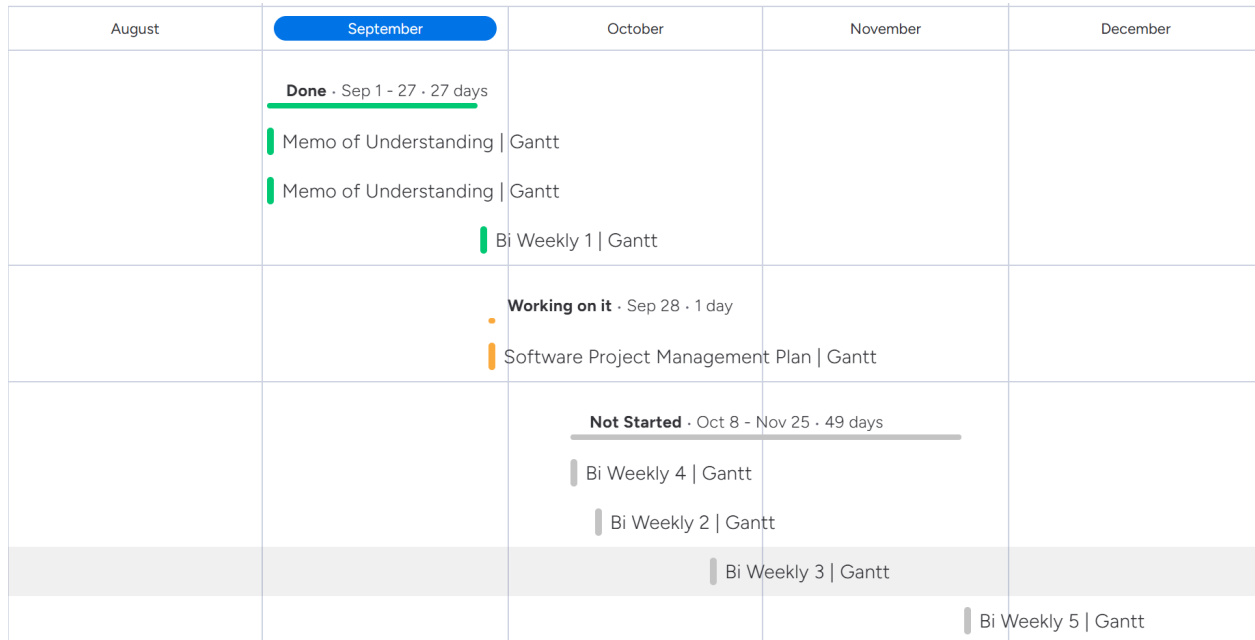
4.3 Control Plan

We will monitor our progress using a Gantt chart and documentation. The gantt chart below shows our current progress which include:

- Current due dates
- Finished tasks
- Tasks in progress
- Unfinished tasks

Each of our tasks are color coded: green means finished, yellow means in progress, and grey means we haven't started on it.

The team meets once every Thursday at 12:30-1:30 PM. Although we only meet once we communicate with one another through GroupMe. The person taking the minutes for our meeting will be Zamiyah. The team will document any issues that we come across so that we can share them with the client, as well as have a form of documentation for our progress.



4.4 Risk Management Plan

During the course of our project there are multiple risks that could present themselves. For example, when exporting or interacting with the code base, the source code for the project could be deleted. Another risk could be a team member not contributing to the project causing a deadline to be missed. While testing, we could come across certain bugs that harm the project. Another risk could be misunderstanding the requirements given by our client. Our team could also lose communication with one another due to an issue with GroupMe or because of issues on campus with class. Our prototype could take much longer to create than expected. Not enough testing could lead to bugs that we were not able to find. The UI design of our project could be confusing to new users.

Number	Risk	Probability	Impact
1	Deleted source code	Low	High
2	Deadline missed	Medium	Medium
3	Critical Bugs	Medium	High
4	Requirements misunderstood	Medium	Medium
5	Communication loss	Medium	Medium
6	Team member unavailable	Low	High

7	Prototype Delay	Medium	High
8	Poor Testing	Medium	High
9	Confusing UI	Low	Medium

4.5 Closeout Plan

Ideally the code will be handed over using the repository which the client would have been invited into. If there were any unfinished tasks by the deadline then we'd communicate it to the client. If they allow us more time to complete the tasks then the team would finish them. Otherwise, we'd save the coding in an archive and document the deliverables and what we could improve upon.

5 Technical Process Plans

5.1 Process Model

Iterative design is when a cycle of creating, testing, and refining is used to create a final product. The project changes and becomes more complete over time. It is heavily associated with the agile lifecycle model. When a project uses the agile lifecycle model, it is developed quickly and frequently changes. Our project incorporates iterative design and the agile lifecycle process.

5.2 Methods, Tools, and Techniques

When using object-oriented design, code and behavior is separated into discrete objects. Each object has its own methods and variables that only it can access. Object-oriented analysis looks at the attributes and behaviors of these objects. We will be programming in Java in Visual Studio Code. Java is used because of its heavy emphasis on object-oriented programming.

Follow these conventions when writing Java code:

- Organize each class in the following order:
 1. Data members
 2. Constructors
 3. Methods
- Class and Interface names should be capitalized
- All alphabetic characters in static final data member identifiers should be upper-case
- Data members should start with a "f_" to easily distinguish them from local variables which start with lower-case alphabetic characters
- Comment data members and constants
- All methods should be commented with Javadoc style comments:
 1. Brief description of what the method does
 2. Description of each parameter value

3. Description of its return values
 4. Exceptions it throws and a brief description of the condition that causes each exception to be thrown
- All method names should start with lower-case alphabetic characters
 - Avoid wrapping lines

5.3 Infrastructure Plan

We will host all our files on GitHub. Programming will be done in Visual Studio Code. Our program will run on any computer that can run Java executables.

5.4 Product Acceptance Plan

- Have word themes
- Randomize themes
- Sub-themes in themes
- Include certain words
- Choose an age group
- Set length of each game
- Customizable difficulty
- Customizable word length
- Length of word list
- Keep it appropriate
- Present two weeks before end of semester

6 Supporting Process Plans

6.1 Configuration Management Plan

Our project will be using GitHub for configuration management and version code. The source code, documentation, and project artifacts will all be stored in the GitHub repository that we have shared. Version control will be conducted through GitHub as they commit their changes on a regular basis. We will ensure that the commit messages have relevance towards what was completed. Our documentation will make sure to follow the same conventions throughout the project. New versions of code and software will be created through our commits on GitHub, along with documents and reports that will be uploaded to the respective repository.

6.2 Documentation Plan

Team FLEX will be responsible for creating and maintaining several documents throughout the lifecycle of our project. These documents include the Memo of Understanding, Software Project Management Plan, Software Configuration Management Plan, Requirements Analysis

Document, System Design Document, Object Design Document, Test Plan, Sprint Review documents, meeting notes, and the Java source code for our program. We will document our source code using JavaDocs. The documents will be updated as our team progresses through the project and will be stored in the shared repository.

6.3 Quality Assurance Plan

Our team will follow the Java coding style that we have been given. Our source code will be documented with JavaDocs. Our quality assurance will include testing of application features throughout the entire project, review of source code from the entire group, ensuring that all our requirements are met, and tracking bugs found or resolved throughout the development of our program. As for user data, the application will not need any sort of sensitive information from our user. With this, stored data will be made sure to be kept to as little as possible. Stored information such as statistics within the application will be protected from malicious modification through secure coding practices.

6.4 Reviews and Audits

We will review the documents and code as a group on a regular basis. Before code or documents are submitted, the team will review the submission to make sure that it is fully completed to the required standards. Code reviews will also be conducted by team members before releases or completions of sprints. We will test as much as possible to verify that our application is functioning properly and the requirements of the project are met. We will also use meetings to review our progress with the milestones we were given to make sure that our development is staying on schedule while also meeting the milestones we need to on time.

7 Additional Plans

We currently have no additional plans.